

Deployment Runbook

From an empty AWS account to a production-style, three-tier microservice streaming platform on Amazon EKS, with monitoring and a Jenkins CI/CD pipeline. Covers operations and teardown.

React 19

Node.js 22

6 microservices

PostgreSQL 16

Redis 7

Docker

Kubernetes / EKS

Helm

Terraform

Jenkins

Prometheus

Grafana

Audience

DevOps / Platform engineers

Target

AWS EKS · us-east-1 (N. Virginia)

Date

3 October 2026

Est. time

~1 hour to first deploy

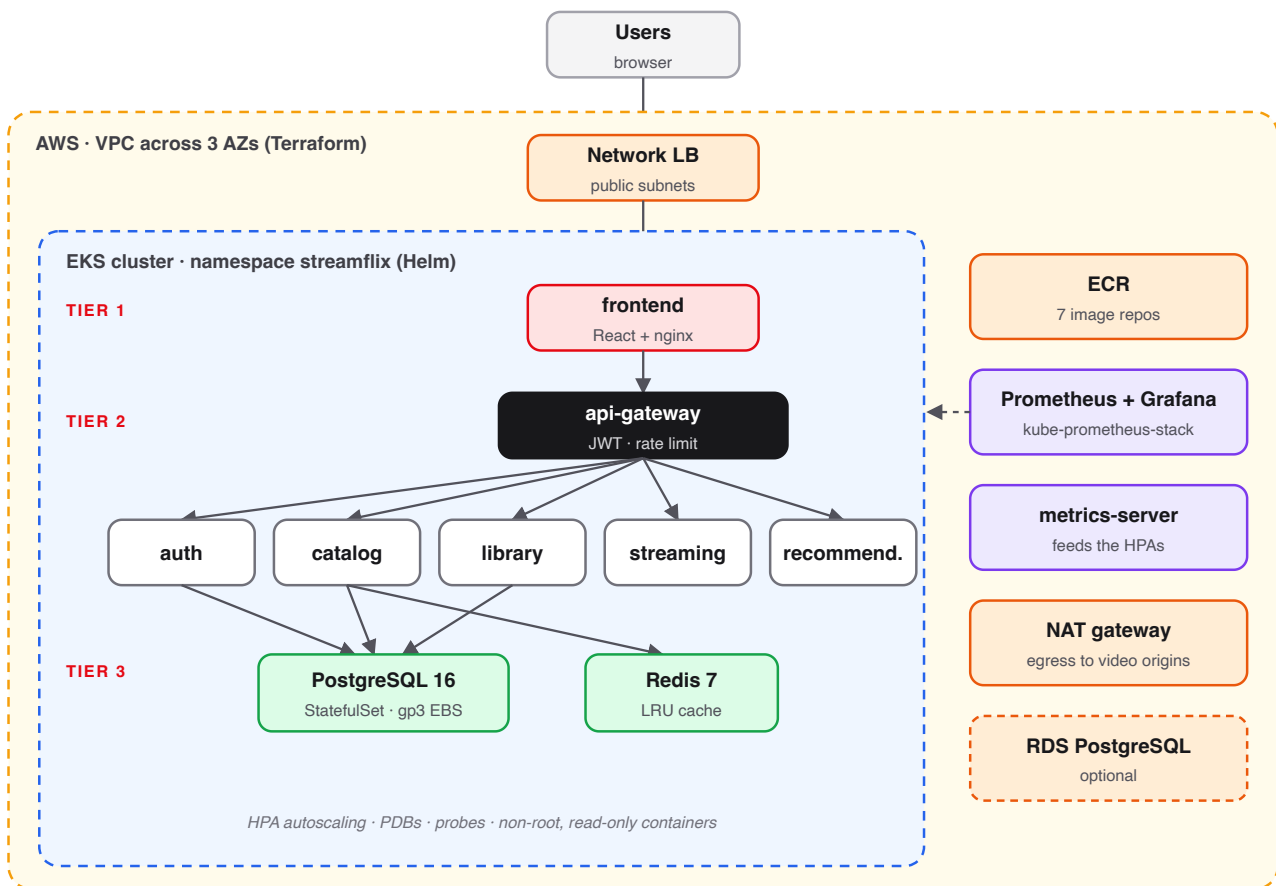
Est. cost

≈ US\$5–7 / day while running

Source

github.com/sanjaybdch9/streamflix

Architecture at a glance



CI/CD flow (Jenkinsfile)



Key facts before you start

7 images

frontend, api-gateway, auth, catalog, library, streaming, recommendation, all tagged with the git commit SHA.

Database per service

auth_db, catalog_db, library_db. Each service creates its own database on first start (in-cluster Postgres or RDS).

Build for linux/amd64

EKS t3 nodes are x86. Apple Silicon builds crash with `exec format error`; `deploy.sh` and Jenkins pin the platform.

Order matters

Run the add-ons (Phase 3) before the first deploy. In teardown, remove the Helm release and volumes before `terraform destroy`.

Phase 0	Prerequisites	tools, AWS account, IAM user, credentials	
Phase 1	Local verification	docker compose (done)	
Phase 2	Infrastructure	terraform → VPC, EKS, ECR	~20 min
Phase 3	Cluster add-ons	metrics-server, Prometheus, Grafana	~5 min
Phase 4	First deployment	build → push → helm → smoke test	~10 min
Phase 5	CI/CD	Jenkins pipeline on every push	
Phase 6	Operate	monitor, scale, roll back, debug	
Phase 7	Teardown	in the right order, so nothing keeps billing	

Cost. With the defaults (EKS control plane, 3 × t3.medium, 1 NAT gateway, 1 NLB, 20 GiB gp3) expect roughly **US\$5–7 per day** while it runs. Do Phase 7 when you finish.

All commands run from the `streamflix/` folder unless stated. Set your region once per terminal:

```
export AWS_REGION=us-east-1
```

Phase 0 — Prerequisites

0.1 Tools on your machine

Tool	Check	Install
Docker Desktop	<code>docker version</code>	<code>brew install --cask docker-desktop</code>
AWS CLI v2	<code>aws --version</code>	<code>brew install awscli</code>
Terraform ≥ 1.6	<code>terraform version</code>	<code>brew install hashicorp/tap/terraform</code>
Helm 3 or 4	<code>helm version</code>	<code>brew install helm</code>
kubectl	<code>kubectl version --client</code>	<code>brew install kubectl</code>

0.2 An IAM user for yourself (never use the root account)

1. AWS Console → **IAM** → **Users** → **Create user**, for example `devops-admin`.
2. Attach the **AdministratorAccess** policy. That's fine for a personal learning account; in a company, use the roles your platform team provides.
3. Open the user → **Security credentials** → **Create access key** → **Command Line Interface**.
4. Save the access key ID and secret. AWS shows the secret only once.

0.3 Configure the CLI

```
aws configure
```

Enter the key, the secret, your region (`us-east-1`) and output format `json` . Then verify:

```
aws sts get-caller-identity
```

It must print your 12-digit account ID. If it says `Unable to locate credentials`, step 0.3 didn't take effect.

Phase 1 — Local verification

Already done. `docker compose up --build` ran all 11 containers healthy on your Mac, so the code and images work. Always re-run this after changes, before you deploy:

```
docker compose up --build -d && curl -s localhost:8080/api/status
```

Stop it before moving on, to free memory: `docker compose down`.

Phase 2 — Infrastructure with Terraform

2.1 (Recommended) Remote state

By default, Terraform state is a local file. If you lose it, Terraform can no longer manage or destroy what it created. For anything beyond a quick test, store the state in S3:

```
BUCKET=streamflix-tfstate-$(aws sts get-caller-identity --query Account --output text)
aws s3api create-bucket --bucket $BUCKET --region us-east-1
aws s3api put-bucket-versioning --bucket $BUCKET --versioning-configuration Status=Enabled
echo $BUCKET
```

Then uncomment the `backend "s3"` block in `terraform/versions.tf` and set `bucket` to the printed name. Versioning lets you recover an earlier state file if one gets corrupted. (In any region other than `us-east-1`, `create-bucket` also needs `--create-bucket-configuration LocationConstraint=<region>`.)

2.2 Configure

```
cd terraform
cp terraform.tfvars.example terraform.tfvars
```

Edit `terraform.tfvars`:

Variable	Set to
<code>region</code>	same as <code>\$AWS_REGION</code>
<code>node_capacity_type</code>	<code>SPOT</code> to save about 70% on a learning cluster
<code>ci_principal_arn</code>	leave commented for now; set it in Phase 5

2.3 Plan and apply

```
terraform init
terraform plan -out tfplan
terraform apply tfplan
```

Read the plan before you apply it: about 70 resources, all new, none destroyed. Apply takes 15–20 minutes, mostly EKS. It creates:

- A VPC across 3 AZs: private subnets for pods and nodes, public subnets for the load balancer, and a NAT gateway.
- An EKS cluster with a managed node group, plus the add-ons CoreDNS, kube-proxy, VPC CNI, Pod Identity and the EBS CSI driver (with its IAM role).
- 7 ECR repositories `streamflix/*`, with scan-on-push and a keep-last-30 lifecycle policy.

2.4 Connect kubectl and verify

```
terraform output          # note ecr_registry and configure_kubectl
aws eks update-kubeconfig --region $AWS_REGION --name streamflix
kubectl get nodes          # 3 nodes, STATUS Ready
kubectl get pods -n kube-system # coredns, aws-node, ebs-csi-* all Running
cd ..
```

Phase 3 — Cluster add-ons (once per cluster)

```
scripts/eks-bootstrap.sh $AWS_REGION streamflix
```

This installs **metrics-server**, which the HorizontalPodAutoscalers need, and **kube-prometheus-stack** (Prometheus, Grafana, Alertmanager and the ServiceMonitor CRD). It prints a Grafana password; save it.

Run this before Phase 4. The chart's EKS values create a `ServiceMonitor`, and the install fails with *no matches for kind ServiceMonitor* if this CRD isn't there yet.

Verify:

```
kubectl top nodes          # numbers, not an error (may take ~1 min)
kubectl get pods -n monitoring # all Running
```

Phase 4 — First deployment

```
scripts/deploy.sh $AWS_REGION streamflix
```

What it does, the same steps as the pipeline:

0. **Refuses to deploy** if there are uncommitted changes or the commit isn't on GitHub, so EKS can always be rebuilt from the repository. For a throwaway experiment, override it with `ALLOW_UNPUSHED=1 scripts/deploy.sh ...`; the images are then tagged `-dirty`.
1. Logs Docker in to ECR.
2. Builds all 7 images for `linux/amd64` and pushes them, tagged with the git commit. EKS nodes are x86, so an image built natively on an Apple Silicon Mac would crash with `exec format error`.
3. Runs `helm upgrade --install` with `values-eks.yaml`. Helm waits for every pod to become ready and rolls back automatically if they don't.
4. Smoke-tests `/api/status` from inside the cluster, then prints the public URL.

Verify like an SRE

```
kubectl -n streamflix get pods           # 14+ pods Running, READY 1/1
kubectl -n streamflix get svc frontend   # EXTERNAL-IP = NLB hostname
kubectl -n streamflix get hpa            # TARGETS show cpu %, not <unknown>
kubectl -n streamflix get pvc            # data-postgres-0 Bound (gp3)
helm -n streamflix history streamflix    # revision 1, deployed
```

Open `http://<EXTERNAL-IP>`, sign up, play a title, then check `http://<EXTERNAL-IP>/status`. The NLB's DNS name can take 2–3 minutes to start resolving.

Make yourself admin (optional)

Admins can add titles through `POST /api/catalog/titles`:

```
helm upgrade streamflix helm/streamflix -n streamflix --reuse-values --set
adminEmails=you@example.com
```

Then sign up with that email.

Phase 5 — CI/CD with Jenkins

Every push then runs: **tests (7 in parallel)** → **helm lint** → **build & push** → **deploy with automatic rollback** → **smoke test**.

5.1 Commit the project

StreamFlix is its own repository: <https://github.com/sanjaybdch9/streamflix>. Jenkins builds whatever is pushed there, so commit and push your changes first:

```
cd ~/Documents/streamflix
git add -A
git commit -m "Describe your change"
git push
```

`.gitignore` already excludes `node_modules/`, `.env`, `dist/` and Terraform state. Check `git status` to confirm none of them are staged.

5.2 Jenkins agent requirements

The agent (or the controller, if it builds itself) needs `docker`, AWS CLI v2, `kubectl`, `helm` and `git`, and the `jenkins` user must be able to run Docker (`sudo usermod -aG docker jenkins`, then restart Jenkins).

5.3 Give Jenkins AWS access

Option A — Jenkins runs on EC2 (best): attach an IAM role to the instance, then skip the credential in 5.4 (leave `AWS_CREDENTIALS_ID` empty).

Option B — anywhere else: create an IAM user `jenkins-ci` with an access key.

Either way, the principal needs these permissions:

- The managed policy **AmazonEC2ContainerRegistryPowerUser**, to push images.
- An inline policy allowing `eks:DescribeCluster` on the cluster, for `update-kubeconfig`.

Then authorise it inside Kubernetes through an EKS access entry. Add its ARN to

`terraform/terraform.tfvars`:

```
ci_principal_arn = "arn:aws:iam::<ACCOUNT_ID>:user/jenkins-ci" # or :role/<instance-role>
```

and apply it: `terraform -chdir=terraform apply`.

5.4 Jenkins configuration

1. **Manage Jenkins** → **Credentials** → **Add**. Kind *Username with password*, ID `aws-credentials`, username = access key ID, password = secret access key (Option B only).
2. **New Item** → **Pipeline** named `streamflix`:
 - Definition: *Pipeline script from SCM*, Git, your repository URL and branch.
 - **Repository URL**: `https://github.com/sanjaybdch9/streamflix.git`, branch `main`.
 - **Script Path**: `Jenkinsfile`
3. **Build with Parameters**: set `AWS_ACCOUNT_ID` (12 digits) and `AWS_REGION`. Keep `APP_DIR` as is unless you moved the project.
4. Optional: add a GitHub webhook (`http://<jenkins>/github-webhook/`) and tick *GitHub hook trigger* so every push deploys.

5.5 Test the pipeline

Make a visible change, for example the hero kicker text in `frontend/src/components/Hero.jsx`. Commit, push and watch the stages. Afterwards `helm -n streamflix history streamflix` shows a new revision with the new image tag.

Phase 6 — Operate

Monitoring

```
kubectl -n monitoring port-forward svc/kube-prometheus-stack-grafana 3000:80
```

Open <http://localhost:3000> (user `admin`, password from Phase 3) → Dashboards → **StreamFlix — Service Overview**. It shows request rate, p95 latency and errors per service, active streams, video egress, cache hit ratio, auth events, memory and event-loop lag.

Everyday commands

Task	Command
Logs of one service (all pods)	<code>kubectl -n streamflix logs -l app.kubernetes.io/name=catalog-service -f</code>
What's running, which version	<code>helm -n streamflix list</code> · <code>kubectl -n streamflix get deploy -o wide</code>
Roll back a bad release	<code>helm -n streamflix history streamflix</code> then <code>helm -n streamflix rollback streamflix <REV></code>
Scale manually	<code>kubectl -n streamflix scale deploy/recommendation-service --replicas=3</code>
Restart a service	<code>kubectl -n streamflix rollout restart deploy/auth-service</code>
Shell into Postgres	<code>kubectl -n streamflix exec -it postgres-0 -- psql -U streamflix -d catalog_db</code>
Watch autoscaling	<code>kubectl -n streamflix get hpa -w</code>

Load test (see the HPA scale out)

```
kubectl -n streamflix run load --rm -i --image=williamyeh/hey -- -z 2m -c 50 http://api-gateway:4000/api/status
```

Watch `kubectl -n streamflix get hpa -w` in another terminal.

Troubleshooting

Symptom	Likely cause → fix
<code>exec format error</code> in pod logs	Image built for arm64. Rebuild with <code>--platform linux/amd64</code> ; <code>deploy.sh</code> and Jenkins already do.
<code>ImagePullBackOff</code>	Wrong registry or tag, or the image wasn't pushed. Run <code>kubectl -n streamflix describe pod <pod></code> and compare with <code>aws ecr list-images --repository-name streamflix/<svc></code> .

Symptom	Likely cause → fix
Services <code>CrashLoopBackOff</code> , logs say <code>database not ready</code>	Postgres not up yet; they retry for 60 s. Check <code>kubectl -n streamflix get pod postgres-0</code> and its PVC.
PVC stuck <code>Pending</code>	EBS CSI driver not running, or <code>gp3</code> StorageClass missing. Check <code>kubectl get pods -n kube-system grep ebs</code> and <code>kubectl get sc</code> .
<code>frontend</code> EXTERNAL-IP stays <code><pending></code>	Public subnets lack the <code>kubernetes.io/role/elb</code> tag (Terraform sets it). Check <code>kubectl -n streamflix describe svc frontend</code> events.
Helm: <i>no matches for kind "ServiceMonitor"</i>	Phase 3 wasn't run. Run <code>eks-bootstrap.sh</code> , or pass <code>--set monitoring.serviceMonitor.enabled=false</code> .
<code>kubectl</code> : <i>You must be logged in to the server (Unauthorized)</i>	Your IAM principal has no EKS access entry. Use the identity that ran Terraform, or add an entry like <code>ci_principal_arn</code> .
HPA shows <code><unknown></code>	metrics-server missing or still starting (Phase 3).
Browser shows "Failed to fetch"	Frontend loaded but the gateway is unreachable. Check <code>kubectl -n streamflix get pods -l app.kubernetes.io/name=api-gateway</code> and <code>/api/status</code> .
Video doesn't play, <code>/api/status</code> is fine	Nodes need outbound internet (NAT) to reach the video origins. Check the streaming-service logs for <code>origin error</code> .

Phase 7 — Teardown (order matters)

Kubernetes creates AWS resources Terraform doesn't know about: the NLB and the EBS volume. Delete those first, or `terraform destroy` hangs on the VPC and leftovers keep billing.

```
# 1. App and its load balancer
helm -n streamflix uninstall streamflix
kubectl -n streamflix delete pvc --all           # Postgres data
kubectl delete namespace streamflix             # also removes the kept Secret

# 2. Monitoring stack
helm -n monitoring uninstall kube-prometheus-stack
kubectl delete namespace monitoring

# 3. Retained EBS volumes (the gp3 StorageClass uses reclaimPolicy: Retain to protect data)
aws ec2 describe-volumes --region $AWS_REGION \
  --filters Name=tag:kubernetes.io/created-for/pvc/namespace,Values=streamflix \
  --query 'Volumes[].VolumeId' --output text
aws ec2 delete-volume --region $AWS_REGION --volume-id <vol-id>

# 4. Wait ~2 minutes for the NLB to disappear, then destroy the infrastructure
terraform -chdir=terraform destroy
```

Finally, check that **EC2** → **Load Balancers**, **EC2** → **Volumes** and **VPC** → **NAT gateways** are empty in your region.

Where each piece lives

Concern	File
Infrastructure	<code>terraform/main.tf</code> , <code>variables.tf</code> , <code>outputs.tf</code>
Cluster add-ons	<code>scripts/eks-bootstrap.sh</code>
Manual build + deploy	<code>scripts/deploy.sh</code>
Kubernetes manifests	<code>helm/streamflix/templates/*</code> , <code>values.yaml</code> , <code>values-eks.yaml</code>
Pipeline	<code>Jenkinsfile</code>
Local stack	<code>docker-compose.yml</code> , <code>.env</code>
Dashboards	<code>monitoring/grafana/dashboards/streamflix-overview.json</code>